Islamic University of Technology

Organization of Islamic Cooperation



Name	**:** Antara Arifa Mullick
ID	**:** 210042132
Program	**:** SWE
Department	**:** CSE
Subject	**:** Artificial Intelligence
Subject Code	**:** CSE 4618
Title	**:** Reinforcement Learning
Date of Submission	**:** 8th September, 2025

1. Introduction

In this lab, the objective was to implement a Q-Learning agent for reinforcement learning environments. Q-Learning is a model-free, off-policy reinforcement learning algorithm that enables an agent to learn the optimal policy through trial and error, without needing a model of the environment. The goal of this experiment was to implement a Q-Learning agent that could learn optimal policies through interactions with different environments such as Gridworld and Pacman. Unlike value iteration methods, Q-Learning allows the agent to learn directly from experience, making it a versatile approach to solving reinforcement learning problems.

2. Problem Analysis

2.1 Core Challenge

The core challenge of implementing Q-Learning was to balance exploration (trying new actions) and exploitation (choosing the best-known action) while ensuring that the agent learns optimal Q-values over time. Key challenges included:

- Handling state-action pairs that the agent has never encountered before, which required initializing their Q-values to 0.0.

- Ensuring that the agent doesn't get stuck in deterministic cycles and continues exploring the environment using random tie-breaking in action selection.

- Implementing the epsilon-greedy strategy to allow for random action selection during exploration and greedy action selection during exploitation.

2.2 Key Implementation Requirements

To implement Q-Learning effectively, the following were necessary:

- **Temporal Difference Learning**: The agent updates its Q-values based on the immediate reward and the discounted future reward.

- **Random Tie-Breaking**: If multiple actions have the same Q-value, the agent randomly chooses one to prevent cycles and promote exploration.

- **Method Abstraction**: Using a uniform approach to retrieve Q-values using the `getQValue()` method ensures the agent's design can be extended in the

future.

3. Solution Explanation

3.1 Core Data Structure

To store the Q-values, I used a Counter (from the util library), which is a dictionary-like data structure. Each state-action pair is stored as a key, and the associated Q-value is stored as its value. The Counter automatically handles previously unseen state-action pairs by initializing them with a default value of 0.0.

3.2 Key Method Implementations

Q-Value Retrieval (getQValue)

```
def getQValue(self, state, action):
  return self.qValues[(state, action)]
```

- This method retrieves the Q-value for a given state-action pair. It utilizes the Counter's default behavior to handle unseen pairs.

Value Computation (computeValueFromQValues)

```
def computeValueFromQValues(self, state):
  legalActions = self.getLegalActions(state)
  if not legalActions:
    return 0.0
  return max(self.getQValue(state, action) for action in legalActions)
```

- This method computes the value of a state by taking the maximum Q-value over all legal actions for that state.

Policy Extraction (computeActionFromQValues)

```
def computeActionFromQValues(self, state):
```

```
    legalActions = self.getLegalActions(state)
    if not legalActions:
        return None
    maxQ = self.computeValueFromQValues(state)
    bestActions = [action for action in legalActions if self.getQValue(state, action) == maxQ]
    return random.choice(bestActions)
```

- This method extracts the action to take from the Q-values by finding the action with the highest Q-value. If multiple actions have the same Q-value, one is randomly chosen.

Action Selection (getAction)

```
def getAction(self, state):
    legalActions = self.getLegalActions(state)
    if not legalActions:
        return None
    if util.flipCoin(self.epsilon):
        action = random.choice(legalActions)
    else:
        action = self.computeActionFromQValues(state)
    return action
```

- This method implements the epsilon-greedy strategy, where the agent explores with probability ε, and exploits with probability 1-ε.

Q-Value Update (update)

```
def update(self, state, action, nextState, reward):
```

sample = reward + self.discount * self.computeValueFromQValues(nextState)

self.qValues[(state, action)] = (1 - self.alpha) * self.getQValue(state, action) + self.alpha * sample

- This method implements the core Q-Learning update rule: $Q(s,a) \leftarrow (1-\alpha)Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a')]$.

4. Findings and Insights

4.1 Learning Behavior Observations

- **Gradual Convergence**: Q-values converged over time, showing steady improvement.
- **Exploration Impact**: Higher epsilon values led to more exploration, slowing down convergence.
- **Learning Wake**: The agent updated its knowledge incrementally, with later states influenced by earlier ones.

4.2 Performance Analysis

- **Initial Phase**: Random actions during exploration resulted in seemingly chaotic behavior.
- **Learning Phase**: Gradual improvement of Q-values as the agent learned the environment's structure.
- **Convergence**: Eventually, the Q-values stabilized, and the agent learned near-optimal policies.

5. Challenges Faced and Solutions

5.1 Challenge: Handling Unseen State-Action Pairs

- **Problem**: The agent needed to deal with unseen state-action pairs during learning.

- **Solution**: The `util.Counter` was used, which handles unseen pairs by assigning them a default value of 0.0.

5.2 Challenge: Tie-Breaking in Action Selection

- **Problem**: Identical Q-values for multiple actions could have caused the agent to behave deterministically.

- **Solution**: I implemented random tie-breaking by selecting randomly among actions with the highest Q-value.

5.3 Challenge: Maintaining Abstraction

- **Problem**: Ensuring code extensibility for future modifications or improvements.

- **Solution**: I ensured abstraction by consistently using the `getQValue()` method to access Q-values.

6. Hyperparameter Exploration

6.1 Learning Rate (α)

- High α (0.8-1.0): Fast learning but instability.

- Medium α (0.3-0.7): Balanced learning speed and stability.

- Low α (0.1-0.3): Slow but stable convergence.

6.2 Epsilon (ε) - Exploration Rate

- High ε (0.3-0.5): Extensive exploration, slower convergence.
- Medium ε (0.1-0.3): Balanced exploration and exploitation.
- Low ε (0.01-0.1): Focus on exploitation, limited exploration.

6.3 Discount Factor (γ)

- High γ (0.9-0.99): Focus on long-term rewards.
- Low γ (0.1-0.5): Immediate rewards are prioritized.

7. Additional Insights and Extensions

7.1 Convergence Guarantees

The Q-Learning agent satisfied the convergence conditions for infinite state-action pair visits and appropriate learning rate.

7.2 Potential Improvements

1. Adaptive epsilon decay for better exploration-exploitation balance.
2. Use of experience replay for more stable learning.
3. Neural networks for function approximation in large state spaces.

7.3 Real-World Applications

Q-Learning can be applied in areas like:

- Game AI development.
- Autonomous vehicle decision-making.
- Resource allocation optimization.

8. Conclusion

This lab demonstrated the implementation of Q-Learning, a crucial reinforcement learning algorithm. Through a process of trial and error, the agent was able to successfully learn optimal policies for various environments. One of the key aspects of the implementation was the effective handling of the exploration-exploitation trade-off, allowing the agent to explore new actions while also exploiting the knowledge it had gained. Additionally, the Q-value updates were managed properly, and the agent's Q-values gradually converged, showing steady progress towards optimal decision-making. Overall, the lab has provided a solid foundation for understanding Q-Learning and has laid the groundwork for exploring more advanced reinforcement learning algorithms in the future.